

DOCUMENTATION TECHNIQUE ETL & DATA QUALITY

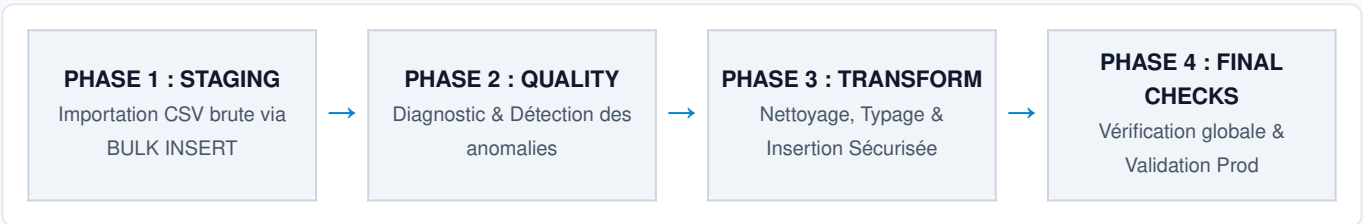
Architecture de pipeline en 4 phases & Spécification des Scripts SQL (Dataset Olist)

Projet : Intégration & Nettoyage ETL Olist
Auteur : Trésor Kitwanda

Cadre : Formation Data Science & Data Engineering
SGBD Cible : Microsoft SQL Server (T-SQL)

1. ARCHITECTURE GÉNÉRALE ET PHILOSOPHIE DU PIPELINE

Dans le cadre de l'ingénierie des données applicative sur le dataset e-commerce Olist, le traitement de la donnée brute suit un workflow rigoureux et linéaire en 4 étapes. L'objectif fondamental est d'assurer la traçabilité complète des données, d'éviter toute perte prématurée d'information lors de l'importation, et d'appliquer des règles d'assainissement systématiques avant d'exécuter l'insertion dans les tables finales de production, suivies d'une validation rigoureuse des contraintes.



Principe Fondamental d'Ingénierie de Données :

Ne jamais appliquer de transformations directement sur les fichiers sources. L'utilisation d'une zone tampon (*Staging Area*) permet de séparer physiquement l'étape d'acquisition de données de l'étape de transformation. La phase 3 réalise le nettoyage et charge directement les tables finales via `INSERT INTO ... SELECT`, laissant la phase 4 exécuter la validation post-chargement pour s'assurer que tout s'est parfaitement déroulé.

2. STRUCTURE ET SPÉCIFICATION DES 4 SCRIPTS SQL

📁 Script 01 : 01_staging_layer.sql (Phase 1 — Acquisition Staging)

Ce script gère la création des tables temporaires `staging_*` ainsi que l'exécution des commandes d'importation massive d'enregistrements (`BULK INSERT`). Les colonnes de staging sont définies de manière permissive (type `NVARCHAR(MAX)`) afin d'absorber la totalité des enregistrements sans risque d'échec de conversion de type lors de la phase initiale.

```
-- Exemple d'importation optimisée avec gestion de séparateur hexadécimal
BULK INSERT staging_sellers
FROM 'C:\Data\olist_sellers_dataset.csv'
WITH (
    FIRSTROW = 2,                -- Ignore la ligne d'en-tête (Header)
    FIELDTERMINATOR = ',',       -- Séparateur de colonnes standard CSV
    ROWTERMINATOR = '0x0a',      -- Saut de ligne UNIX/Linux (LF) en hexadécimal
    CODEPAGE = '65001',          -- Encodage strict UTF-8 (Caractères accentués)
    TABLOCK,                     -- Verrouillage de table (Minimal Logging)
    MAXERRORS = 0                 -- Arrêt immédiat en cas d'erreur de structure
);
```

- **Gestion du ROWTERMINATOR = '0x0a'** : L'usage du code ASCII hexadécimal (0x0a pour LF ou 0x0d0a pour CRLF) élimine les ambiguïtés d'interprétation entre systèmes d'exploitation (Linux/Windows).

- **Gestion des textes complexes (FORMAT = 'CSV') :** Pour les tables contenant du texte libre (ex: staging_order_reviews), l'option FORMAT = 'CSV' est appliquée pour traiter correctement les sauts de ligne ou virgules incluses au sein des commentaires entre guillemets.

Script 02 : 02_data_quality_staging.sql (Phase 2 — Diagnostic & Data Quality)

Ce script applique une batterie de tests non-destructifs sur la zone de staging pour identifier et quantifier les anomalies avant tout traitement.

- **Contrôle des volumes :** Analyse du nombre de lignes via `SELECT COUNT(*)` pour valider l'exhaustivité de l'importation.
- **Détection des valeurs NULL ou vides :** Identification des champs obligatoires manquants grâce à des agrégations conditionnelles :
`SUM(CASE WHEN col IS NULL OR TRIM(col) = '' THEN 1 ELSE 0 END).`
- **Détection des doublons sur clés candidates :** Utilisation de la clause `GROUP BY ... HAVING COUNT(*) > 1.`
- **Contrôle des formats régionaux :** Vérification de la longueur des codes d'État (`LEN(TRIM(seller_state)) <> 2`).

📁 Script 03 : 03_transformations.sql (Phase 3 — Nettoyage, Transformation & Insertion Sécurisée)

Ce script réunit la logique métier de transformation des données brutes stockées en staging ET leur chargement sécurisé dans les tables finales cibles de production via l'instruction `INSERT INTO ... SELECT ...`.

- **Nettoyage et Standardisation** : Suppression des espaces superflus avec `TRIM()` et mise en majuscules systématique des codes d'État avec `UPPER()`.
- **Conversion de Types (Transtypage)** : ** Conversion explicite des chaînes de caractères vers les types physiques cibles (`DATETIME` pour les horodatages, `DECIMAL(10,2)` pour les montants financiers).
- **Imputation des Valeurs Manquantes (Handling NULLs)** : **
 - Remplacement des titres et commentaires de reviews absents par `'NO_TITLE'` et `'NO_COMMENT'`.
 - Remplacement des catégories de produits non renseignées par la constante `'NON_SPECIFIE'`.
- **Insertion Sécurisée** : Exécution directe de l'insertion nettoyée dans les tables finales pré-crées avec contraintes physiques de clés.

📁 Script 04 : 04_final_layer_checks.sql (Phase 4 — Vérification & Validation Post-Chargement)

Dernier script du pipeline dont le rôle est de tester la table finale de production après le chargement pour vérifier que tout s'est déroulé sans anomalie ni déperdition.

- **Contrôle de Non-Perte de Données** : ** Comparaison finale des volumes entre `staging_*` et tables finales de production (`COUNT(*) staging vs COUNT(*) final`).
- **Validation d'Unicité** : ** Contrôle post-insertion des clés primaires simples (ex: `customer_id`) et composées (ex: `order_id + order_item_id`).
- **Validation d'Intégrité Référentielle** : ** Exécution de requêtes `LEFT JOIN ... WHERE NULL` pour s'assurer qu'aucune commande orpheline n'existe sans client, produit ou vendeur associé.
- **Validation des Règles Métier Finales** : ** Contrôle de la cohérence chronologique (`order_purchase_timestamp <= order_estimated_delivery_date`) et de la non-négativité des prix (`price >= 0`).

3. MATRICE MÉTIER DU PIPELINE PAR TABLE

Table	Phase 1 : Staging (Bulk)	Phase 2 : Quality (Diagnostic)	Phase 3 : Transform & Insertion	Phase 4 : Final Verification
CUSTOMERS	<code>staging_customers</code>	Détection doublons <code>customer_id</code>	Nettoyage <code>TRIM</code> + <code>INSERT INTO customers</code>	Vérification <code>COUNT(*)</code> & Unicité PK
SELLERS	<code>staging_sellers</code> (<code>ROWTERMINATOR='0x0a'</code>)	Contrôle longueur <code>seller_state <> 2</code>	<code>UPPER(seller_state)</code> + <code>INSERT INTO sellers</code>	Contrôle format ISO final & zéro NULL
PRODUCTS	<code>staging_products</code>	Détection dimensions & poids manquant	Imputation <code>'NON_SPECIFIE'</code> + <code>INSERT INTO products</code>	Vérification absence de NULLs résiduels
ORDERS	<code>staging_orders</code>	Détection des inversions de dates	<code>CAST</code> vers <code>DATETIME</code> + <code>INSERT INTO orders</code>	Validation FK <code>customer</code> & dates cohérentes

Table	Phase 1 : Staging (Bulk)	Phase 2 : Quality (Diagnostic)	Phase 3 : Transform & Insertion	Phase 4 : Final Verification
ORDER_ITEMS	staging_order_items	Vérification prix < 0	Typage DECIMAL + INSERT INTO order_items	Vérification unicité clé composée (order, item)
PAYMENTS	staging_order_payments	Détection séquences doublonnées	Typage + INSERT INTO order_payments	Validation FK order_id & totaux exacts
REVIEWS	staging_order_reviews (FORMAT='CSV')	Détection textes manquants	Imputation NO_TITLE + INSERT INTO order_reviews	Vérification schéma & liaisons orphelines

4. CONCLUSION ET BONNES PRATIQUES IMPLÉMENTÉES

La méthodologie en 4 phases ainsi ajustée respecte scrupuleusement le cycle de vie applicatif d'un pipeline ETL professionnel. En exécutant le nettoyage et l'insertion dans la Phase 3 (Transformation), puis en isolant l'étape d'audit et de contrôle dans la Phase 4 (Final Checks), nous garantissons que chaque enregistrement présent dans la base finale a été à la fois correctement transformé et formellement validé post-chargement.